

University of Reading
Department of Computer Science

Predictive Analysis of Professional Counter-Strike Player Performance Using Historical HLTV Statistics

Gabriel Dorosh

Supervisor: Ferran Espuny-Pujol

A report submitted in partial fulfilment of the requirements of
the University of Reading for the degree of
Bachelor of Science in *Computer Science*

May 15, 2026

Declaration

I, Gabriel Dorosh, of the Department of Computer Science, University of Reading, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of UoR and public with interest in teaching, learning and research.

Gabriel Dorosh
May 15, 2026

Abstract

This project investigates whether historical statistics can predict upcoming professional Counter-Strike 2 player-map performance. Professional matches produce rich performance data that can be analysed. Esports teams, analysts, and spectators are all incentivised to analyse this data to estimate future performance.

This project's dataset contains 63,010 player-map rows, made up of games played by 1,795 professional players, all collected from the public HLTV database. The chosen target, `good_match`, is defined as `good_match = rating > 1.0`.

Historical features were constructed using chronological ordering and shifted calculations to reduce the risk of data leakage. They were constructed using historical data relating to player performance, map-specific performance, team form, and opponent context. Random splits were explicitly not used. The models were selected using validation and evaluated on the future test period. The test period had previously been used as part of baseline development. Therefore, the test results are not fully independent evidence but indicative. The dataset lacks richer contextual data such as roles, economy, utility usage, tactical strategies, or other demo-level information.

The validation-selected soft-vote classifier reached test ROC-AUC 0.630. The secondary regression experiment yielded a test $R^2 = 0.083$ with the validation-selected regressor. The models provided limited evidence of modest above-baseline ranking signal when tested on the chronological evaluation period. They do not support accurate upcoming player-map predictions.

The main contribution of the project is a leakage-aware pipeline that tests the limits of evaluating future player performance from public summary data.

Keywords: esports analytics, Counter-Strike, HLTV, machine learning, player performance prediction

Report's total word count: 14,325

Acknowledgements

I would like to thank my supervisor for helpful guidance throughout, particularly in helping to keep the work focused on a clear research question and an appropriate final-year scope.

In addition, I would like to thank my family and friends for their encouragement during the project.

Contents

Abstract	ii
Acknowledgements	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Question, Aim and Objectives	2
1.4 Solution Approach	3
1.5 Summary of Contributions and Achievements	3
1.6 Organisation of the Report	4
2 Literature Review	5
2.1 Esports Analytics Context	5
2.2 Prediction in Sport and Esport	5
2.3 Existing Esports Prediction Methods	6
2.4 Counter-Strike Statistics and Performance Metrics	6
2.5 Machine Learning for Tabular Prediction	7
2.6 Evaluation Metrics and Temporal Validation	7
2.7 Legal, Social and Ethical Issues	8
2.8 Literature Gap	8
3 Methodology	10
3.1 Research Design	10
3.2 Data Source and Collection	10
3.3 Data Cleaning and Target Construction	11
3.4 Feature Engineering	12
3.5 Modelling Design	13
3.6 Train, Validation and Test Split	15
3.7 Evaluation Metrics	15
3.8 Validation Checks	16
3.9 Implementation and Reproducibility	17
3.10 Social, Legal and Ethical Considerations	18
3.11 Chapter Summary	19

4	Results	20
4.1	Dataset Summary	20
4.2	All-Player Baseline Models	21
4.3	Extended Classification Experiments	21
4.4	Regression Results	22
4.5	Summary of Findings	23
5	Discussion and Analysis	24
5.1	Interpretation of the Predictive Signal	24
5.2	Why Prediction is Difficult	24
5.3	Classification and Regression Compared	25
5.4	Was the Dataset Large Enough?	26
5.5	Model Behaviour and Feature Design	26
5.6	Limitations and Threats to Validity	27
5.7	Ethical Interpretation	29
5.8	Overall Answer to the Research Question	29
6	Conclusions and Future Work	31
6.1	Conclusion	31
6.2	Future Work	32
7	Reflection	33
7.1	Project Development	33
7.2	Learning About Data Leakage	33
7.3	Reproducibility and Scope Control	34
7.4	Personal Evaluation	34
	References	36
	Appendices	39
A	Feature Groups	39
A.1	Core Pre-Map Features	39
A.2	Map-Known Features	40
A.3	Starting-Side-Known Features	40

List of Figures

3.1 Pipeline from page collection and historical feature construction to chronological model selection and final model evaluation.	10
--	----

List of Tables

2.1	Comparison of related esports analytics studies with this research task.	9
3.1	Main raw data groups collected from public HLTV player-map pages.	11
3.2	Feature availability tiers and the assumptions made of available information. .	13
3.3	Model families and configurations evaluated during validation.	14
3.4	Validation checks used during feature dataset construction.	16
3.5	Project components and scripts.	17
3.6	Social, legal and ethical risks, implemented mitigations and residual limitations.	18
4.1	Processed dataset summary.	20
4.2	Baseline all-player classification results on the chronological test set.	21
4.3	Selected extended classification experiment results with comparisons.	22
4.4	Selected soft-vote classifier confusion matrix at threshold 0.50.	22
4.5	Selected rating regression results on the chronological test set.	23
5.1	Main limitations and validity threats.	28

List of Abbreviations

ADR	Average damage per round
CS2	Counter-Strike 2
CS:GO	Counter-Strike: Global Offensive
CSV	Comma-separated values
CT	Counter-Terrorist
EWM	Exponentially weighted mean
FPS	First-person shooter
HLTV	A public Counter-Strike news and statistics website
IGL	In-game leader
KAST	Percentage of rounds in which a player had a kill, assist, survived, or was traded
KDR	Kill-to-death ratio
MAE	Mean absolute error
RMSE	Root mean squared error
ROC–AUC	Area under the receiver operating characteristic curve
T	Terrorist

Chapter 1

Introduction

1.1 Background

[Hamari and Sjöblom \(2017\)](#) define esports as a competitive and sport-like activity mediated through electronic systems, with players, teams, and spectators interacting through computer interfaces. Esports has become a major global entertainment and sporting market.

Counter-Strike has a rich history and the professional scene has since grown into an international industry. Counter-Strike 2 (CS2), released by Valve in 2023 as the successor to Counter-Strike: Global Offensive (CS:GO) in 2023, continues Counter-Strike's core format of being a five-versus-five tactical first-person shooter (FPS) in which teams compete over a competitive map pool ([Valve, 2026](#)). Outside of the core format, there have been many updates and versions over the years with changes to the economy, MR format, mechanics, and maps.

HLTV publicly publishes professional Counter-Strike match and player statistics ([HLTV.org, 2026](#)). Using their summary data for CS2 predictions has the issue of important context such as role labels, round-to-round economy, tactics, and preparation information being unavailable.

In traditional sport, machine learning has already been applied to tasks such as match outcome prediction, player evaluation, and tactical analysis ([Bunker and Thabtah, 2019](#)). One of the key issues with CS2 analysis is that important contextual data is missing from HLTV's database, such as player roles, team economy, strategy, preparation, roster changes, and game patches/changes. These impact a player's performance, but are not being tracked publicly in public HLTV statistics.

Professional Counter-Strike teams are increasingly using dedicated analysts and data-driven review to help with decisions relating to their rosters, with analytics aiding scouting, player development, and anti-stratting ([Xenopoulos et al., 2022](#)). Despite these advances, video and demo review remain important to analysts. Future performance predictions would benefit analysts, and in turn also benefit spectators through media analytics used to explain narratives.

The project turns the task of finding future player-map performance into binary classification. A player-map with a HLTV rating greater than 1.0 is defined as a `good_match`. A rating above 1.0 is commonly used as a threshold for an above average performance, but is not a complete overview of player quality ([Milovanovic, 2017](#)).

1.2 Problem Statement

HLTV rating is an post-map summary that is intended to distinguish between above- and below-average output, designed to combine several aspects of map statistics ([Milovanovic,](#)

2017). Ratings are useful as a summary of performance, but there are issues that arise when comparing player ratings directly. The threshold `rating > 1.0` is used within this project as an operational definition of `good_match`, but it is not a definitive definition of player output in a map. Player rating for a specific map is affected by more than individual skill and form. The context surround a rating is important, the team's form could be the exact same between games but by only looking at their rating it can easily be misinterpreted.

The model must predict upcoming `good_game` using only information available before the start of the target map. The datasets and features must be designed in such a way that the risk of target and temporal leakage is reduced, as to produce fair results. Leakage would change the task from predicting an upcoming player-map outcome, to explaining or reconstructing it.

The first challenge is collecting usable match data. HLTV is a service that provides public Counter-Strike statistics. Relevant pages have to be scraped and cached to collect the map statistics, and then parsed into structured datasets. Within the dataset, every row represents a single player's performance within a single map. Each map normally produces ten player-map rows, complete with their statistics for that map.

The second challenge is taking steps to reduce data leakage. If there were data leakage, the question the model is answering goes from will it be a good game, to was it a good game, which is much easier to answer. To ensure the model is answering the correct question, all features used for prediction must use only historical data. It becomes important that time order is respected; a normal random train/test split would not be appropriate as future matches would be included in the training. A chronological split must be used. Shifting historical aggregate calculations also ensures the row for a map must only use rows from previous maps.

The project test for any reliable, predictive above-baseline signal in public, historical data. Weak signal would still be useful as it would identify a relationship worth investigating, and it highlights the limitations of this approach of using public data and the shortcomings from not having full context. Weak signal is not enough to justify reliable individual prediction.

1.3 Research Question, Aim and Objectives

The research question is:

Can a professional Counter-Strike player's historical match data predict good upcoming professional map performances on Counter-Strike 2?

The project predicts player-map performance, not match winners, series winners, or individual round outcomes. The aim of the project is to design and evaluate a machine learning pipeline for predicting upcoming professional Counter-Strike player performance from public player statistics.

The objectives are to:

- Scrape, cache, and parse HLTV map statistics pages into a structured dataset,
- Engineer historical features,
- Separate features into time availability tiers,
- Select classification models using a validation chronological training/validation/test split,
- Experiment with a secondary regressor to assess whether the selected binary target is too coarse,

- Evaluate selected model results against against dummy baselines and interpret their limitations.

1.4 Solution Approach

A custom pipeline was created to download and cache public HLTV result, match, and map-stat pages. The reason pages were cached is so that they could be reused and skipped over when re-running the scraper, this both reduced load and sped up re-runs. The raw page data was then parsed into a structured dataset of player-map rows, each described player performance on a given map with a player and map-stat identifier.

Historical features were engineered to provide recent form, workload, map, opponent, and team context. To ensure reduction in leakage-risk, rows were ordered chronologically and any rolling or expanding outcome calculations were shifted so that each row intentionally used only past data. The features were then grouped by time availability into core, map-known, and starting-side-known tiers.

The project was initially training one model per player which left many observations within the dataset unused and yielded limited player-dependent results. The final modelling approach trained on all player-map rows and resulted in much more of the dataset being used. It allowed the model to find shared patterns across players and be less dependent on one player's history. The target variable `good_match` is defined per row as any row with `rating > 1.0`. The main task remains binary classification of `good_match` with continuous-rating regression existing as a secondary comparison analysis.

The models were evaluated using chronological training/validation/test splits. The classification selection metric was chosen to be validation ROC-AUC, and the regressor selection metric was chosen to be validation RMSE. Further metrics accuracy, balanced accuracy, precision, recall, F1 and confusion matrix counts were included and reported to help interpretation, but not for model selection. For both models, dummy baselines were used as a comparison. The classification model used a majority class dummy, and the regression model used a mean prediction dummy.

1.5 Summary of Contributions and Achievements

The project implemented a custom pipeline for collecting and parsing Counter-Strike player-map statistics from HLTV. It developed a reusable feature-engineering module with leakage-aware feature groups and shifted rolling, expanding, map, opponent, workload, and team histories. It had an evaluation design that separated models by feature availability tiers. It provided a critical comparison of how much predictive signal is present in the classification and regression models when tested in the chronological evaluation, compared to their dummy baselines.

The final dataset featured 63,010 player-map rows from a total of 1,795 players between the 14th of January 2016 and the 1st of March 2026. The positive class rate for the chosen `good_match` threshold within the dataset was close to balanced at 52.8%. The validation-selected classifier was a soft-vote ensemble that achieved test ROC-AUC 0.630, whilst the validation-selected regressor had a test $R^2 = 0.083$. These results show that historical HLTV statistics provided limited evidence of at least some predictive information about future player-map performance on the selected test period.

The project demonstrated a careful, reproducible and leakage-aware pipeline that can be extended in future with richer contextual data. It also showed and reported the existing

potential and limitations of current public data.

1.6 Organisation of the Report

Chapter 2 contains the literature review, including esports analytics, traditional sports prediction, machine learning methods for tabular data and ethical issues around public data.

Chapter 3 discusses methodology, including the data-source, collection pipeline, feature engineering design, leakage-risk reduction, modelling methods and evaluation metrics.

Chapter 4 presents the results.

Chapter 5 interprets the results.

Chapter 6 concludes the project and reflects on future work.

Chapter 7 reflects on the project process.

Chapter 2

Literature Review

2.1 Esports Analytics Context

Esports can be looked at as competitive activity in which the sporting interaction is mediated by electronic systems, with play, competition, and spectating occurring through computer interfaces ([Hamari and Sjöblom, 2017](#)). As matches occur entirely inside software systems, it makes it possible for analysts to create detailed digital event logs that support quantitative analysis.

Professional esports organisations are increasingly following traditional sports by hiring dedicated analysts and using data-driven review processes for their players. However, video and demo review still remain an important part of the post-map review process. ggViz shows how newer Counter-Strike telemetry and specialist analytical tools are able to support analysts and can help players and staff look deeper into player and team performance rather than replacing expert review ([Xenopoulos et al., 2022](#)).

Counter-Strike is suitable for this project as it has a deep rooted and long standing professional scene and accessible data. HLTV stores and provides accessible recorded match and player statistics on a public database which gives sufficient data to build a player-map dataset that meets the requirements for this project. HLTV however does not provide a complete picture and misses out on context such as role labels, tactics, economy, and preparation. Demo-derived data is shown by ESTA to be capable of construct into features with substantially richer context at the cost of different granularity and collection cost ([Xenopoulos and Silva, 2022](#)). Privately collected data could also provide a much more complete picture of the context.

2.2 Prediction in Sport and Esport

Traditional sports prediction research often look at historical performance, recent form, home advantage, team quality, rankings and ratings, of not just the team playing, but the opponents too because outcomes arise from interacting factors [Bunker and Thabtah \(2019\)](#). A lot of these ideas can be applied to esports. In CS2 specifically, player rating, ADR, opening duel performance, map specific performance, and historical performance can be looked at for all of these features. The features chosen within this project are all motivated by this idea of contextualising a player's form, but individual esports player performance adds dependence to role and team contexts that are less visible in aggregate statistics.

[De Bois et al. \(2025\)](#) presents a role-specific individual performance and skill-rating framework designed for use within professional League of Legends esports. Although League of Leg-

ends roles do not directly transfer to Counter-Strike, this supports the notion that role-specific context is important for prediction. Where they have every role modelled independently, this project is role-agnostic as it lacks a reliable source of Counter-Strike role labels.

[Smerdov et al. \(2020\)](#) predicts moment-level player performance using a combination of physiological, environmental, and chair-sensor data from professional and amateur players. Their work supports individual esports performance prediction, but its CS:GO deathmatch context and live sensor inputs differ to this project's professional map-level prediction setting and reliance on public historical data.

These studies show that with the right context, individual esports player performance can be modelled. They also highlight this project's limitation of having less intrusive and more reproducible public data that contains substantially less context.

2.3 Existing Esports Prediction Methods

Similarly to traditional sports, much of esports prediction literature focuses on team or match outcomes. For example, [Hodge et al. \(2021\)](#) evaluated live professional Dota 2 winning team prediction using team-level information collected after a match starts. This contrasts with this project's approach of individual-level professional prediction using inputs that are historical at prediction time.

[Petri et al. \(2021\)](#) analysed more than 3,500 professional Counter-Strike matches and over 25,000 map-selection decisions. It modelled team map-vetoes in professional Counter-Strike as a contextual decision problem and estimated how they affect predicted map win probability. It found that the map picks and bans policies implemented by teams were not always optimal. It showed that the map-veto process contains meaningful pre-match context, although it was looking at how it can affect predicted win probabilities.

This project does not predict the match winner, it is looking at the narrower question of individual performance. It asks whether public historical summary data can rank upcoming individual player-map performance outcomes before the start of the target-map. This gives the project a different granularity and horizon and makes it more relevant to analytically limited areas such as scouting, individual player analysis, and performance monitoring.

2.4 Counter-Strike Statistics and Performance Metrics

Common Counter-Strike performance measures are player kills, deaths, assists, ADR, KAST, opening kills/deaths, clutches, and most importantly for this project, rating. Rating is useful as it summarises many aspects of player performance and can be easily turned into a clear binary target, though it is not a complete measure of performance. The formula behind rating has also changed over the timespan of this dataset with Rating 2.0 being introduced in 2017, 2.1 in 2024, and 3.0 in 2025 ([Milovanovic, 2017](#); [Richards, 2024, 2025](#)). Whilst, within this project, a `good_match` is defined as a player having a rating on a map greater than 1.0, the construct behind rating is not consistent throughout Counter-Strike eras.

Using over 70 million in-game events, [Xenopoulos et al. \(2020\)](#) proposed a context-aware framework that instead values players' actions through their changes to the team's win probability. This displays how action-level contexts are able to capture context not available in map-level summaries. Richer context would address one of the limitations of using a single aggregate rating. More granular data is technically obtainable from map demos. [Xenopoulos and Silva \(2022\)](#) were able to extract 8.6 million actions, 7.9 million game frames, and 417,000 trajectories from a total of 1,558 professional Counter-Strike demos. This contrasts with the

map-level summaries used within this project and indicates a clear path for obtaining richer features in future work.

The literature suggests future performance prediction requires temporal feature engineering and rich context. Traditional sports-prediction research supports the use of recent-form and opponent features (Bunker and Thabtah, 2019), while Counter-Strike roster research showed that both collective and individual performance initially declined when a team experiences roster changes before gradually recovering (Zhang et al., 2022). This supports the use of temporally ordered player, team, and opponent histories, but also shows how this project’s public summaries are incomplete by not including roster or role changes.

2.5 Machine Learning for Tabular Prediction

The modelling dataset for this project is tabular, with each row representing one player-map performance observation and each column within a row representing a feature or target. Dummy baselines provided no-skill baselines, while logistic regression and Ridge regression provided linear reference models. Random Forests, Extra Trees, and Gradient Boosting are able to model interactions, non-linear relationships and feature interactions (Breiman, 2001; Geurts et al., 2006; Friedman, 2001). They allow the project to test whether lightly tuned tree ensembles improve on simpler references without the assumption that the most complex model was the most superior.

Player, team, and opponent identifiers are high-cardinality categorical variables. Target encoding replaced each category with a target-derived estimate. Micci-Barreca (2001) provided the early smoothed target-encoding approach that shrunk sparse category estimates toward a broader mean. Pargent et al. (2022), a later benchmark, supported regularisation for high-cardinality variables. Their approach achieved competitive performance across multiple algorithms and high-cardinality datasets, whilst also showing the value of cross-fitted regularisation. These studies directly support smoothing in this project, but the smoothing value chosen in this project (50) was a project design choice rather than a value established from those studies.

The project drew comparisons between one-hot linear models, numeric tree ensembles, and the selected smoothed target-encoded tree models. A secondary continuous-rating regression experiment was also evaluated so it could be compared against classification to see whether a stronger relationship hid behind the binary threshold.

2.6 Evaluation Metrics and Temporal Validation

Classification performance can be evaluated using accuracy, balanced accuracy, precision, recall, F1, ROC–AUC, or confusion matrix counts. Accuracy is less likely to be distorted by majority-class distortion as the class distribution is roughly balanced within this project, but accuracy still only describes one decision threshold. ROC–AUC was chosen for classifier selection as it evaluates the ranking ability across thresholds. Threshold-dependent metrics are still useful as they describe the false positives and negatives produced by particular operating points. Validation RMSE was used as the main evaluation metric for regression candidates as it measures average rating error and punishes larger rating errors, which is highly interpretable. Both MAE and R^2 were additionally used and reported for additional help in interpretation.

Esports prediction also faces the issue of temporal change as updates typically make changes to game rules, mechanics, and meta balances. Chitayat et al. (2023) studies patch-agnostic Dota 2 esports analytics and highlights how frequent game updates can shorten the

useful lifespan of a predictive model. This relates to this project as the dataset used spans roughly ten years and covers both Counter-Strike: Global Offensive and Counter-Strike 2.

Temporal leakage is a central methodological risk to this project. If there were any temporal leakage then it would not matter which model was chosen, as data leakage can create substantially overoptimistic machine-learning results and can be difficult to detect through headline metrics ([Kapoor and Narayanan, 2023](#)). This is why steps such as temporal ordering, current-row exclusion, and separation of training, validation, and test splits are crucially important to this project.

For non-stationary temporal data, evaluation procedures that preserve observation order. [Cerqueira et al. \(2020\)](#) found repeated out-of-sample testing periods are able to provide a more realistic estimate than random mixing of observation chronology. This project's task is not conventional univariate time-series forecasting but it still carries the risk that performance depends on the chosen era. This supports this project's chronological splits whilst also showing the weakness of a singular holdout period against repeated temporal evaluation.

2.7 Legal, Social and Ethical Issues

The data this project uses is public esports data sourced from professional games. The data being publicly accessible does not automatically remove any data-protection considerations. Player names, in-game usernames, and performance records can all constitute personal data where they relate to identifiable living people ([Information Commissioner's Office, n.d.b](#)). The data collection was limited to just the fields necessary for the stated purposes of this project's research, consistent with the principle of data minimisation ([Information Commissioner's Office, n.d.a](#)). As the data relates to living people, the results must be used cautiously as to not harmfully profile a player.

The biggest risk is that the predictive models can affect how a player is judged or perceived. Player names should be handled carefully as the project could undervalue a player whose role or team contribution is not accurately measured in the available statistics. The lack of complete context should also be mentioned as a limitation of this project alongside its outputs. The scrape-based data collection creates a responsibility of the project to minimise load and attribute the source, how this was done is reported in Section [3.10](#).

2.8 Literature Gap

Comparisons between the prediction target, granularity, and information of the closest related studies are shown in Table [2.1](#).

These comparisons reveal four neighbouring research strands. [Hodge et al. \(2021\)](#) address live team-level winner prediction; [Petri et al. \(2021\)](#) address pre-match map-veto decisions; [Xenopoulos et al. \(2020\)](#) use demo-derived actions for retrospective valuation or granular datasets; and [Smerdov et al. \(2020\)](#) and [De Bois et al. \(2025\)](#) model individual performance using sensor or role information. These works all stand to establish relevant Counter-Strike analytics methods, yet none directly address the same task as that faces this project.

The gap addressed by this project is not general esports prediction, but the narrower question of whether individual player pre-map forecasting is supported using only public historical Counter-Strike summary statistics. The contribution is the combination the custom public-data pipeline, shifted historical features, chronological model selection and information-availability tiers. This project provides a scoped contribution rather than claiming that individual esports performance has previously never been modelled.

Table 2.1: Comparison of related esports analytics studies with this research task.

Study	Game and granularity	Outcome or contribution	Difference to this project
Hodge et al. (2021)	Dota 2, live team features	Live match winner	Not historical individual pre-map prediction
Petri et al. (2021)	CS:GO, map-veto decisions	Map choice optimisation	Not performance prediction
Smerdov et al. (2020)	CS:GO, deathmatch plus sensors	Moment-level player status	Requires private physiological and environmental data
Xenopoulos et al. (2020)	CS:GO, demos and action trajectories	Retrospective action value	Not a public summary pre-map target
Xenopoulos and Silva (2022)	CS:GO, demos and action trajectories	Public granular dataset and win-prediction benchmarks	Not a public-summary pre-map individual rating target
De Bois et al. (2025)	League of Legends	Role-specific individual performance and skill ratings	Different game, role structure, and purpose
This project	CS:GO and CS2, public map summaries	Upcoming player-map rating threshold	Historical, public-data, pre-map individual forecasting

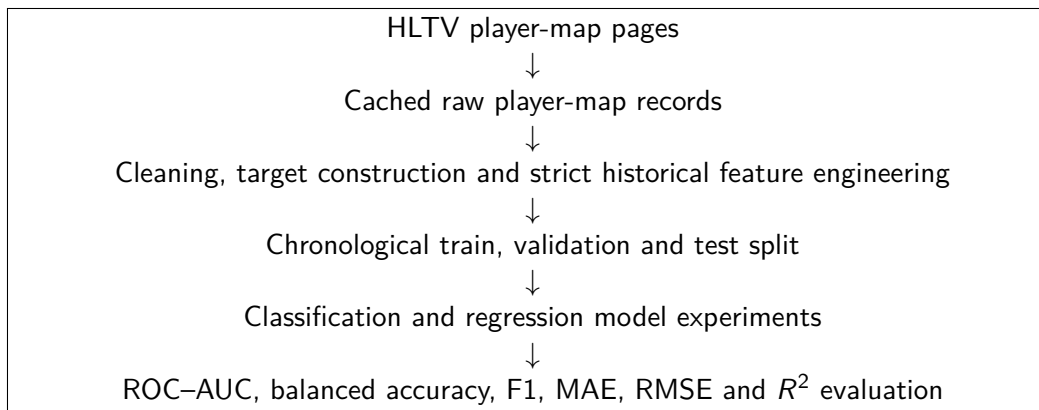
Chapter 3

Methodology

3.1 Research Design

This project was conducted as an applied machine-learning study to answer the question, "Can a professional Counter-Strike player's historical match data predict good upcoming professional map performances on Counter-Strike 2?". The methodology followed the pipeline made in Python which is summarised below in Figure 3.1. Public HLTV pages were collected and cached locally, parsed into player-map rows, cleaned and converted into strictly historical features, divided into a chronological training/validation/test split, different models were compared with each other, candidate selection was done using validation ROC-AUC and RMSE, and the chosen model was evaluated against a more recent chronological period.

Figure 3.1: Pipeline from page collection and historical feature construction to chronological model selection and final model evaluation.



The project design focused on ensuring realistic prediction-time validity to ensure the project does not produce inflated performance results. Both the collection and parsing were handled by a custom public-date pipeline complete with leakage-aware historical feature construction, three information-availability tiers, and chronological evaluation. The feature-availability tiers test the strength of the additional context to see if it contains useful signal.

3.2 Data Source and Collection

This project's data source has been HLTV, a public website containing a vast database of professional Counter-Strike match statistics. From this website, match statistics were collected

and parsed at a player-map level, so every row within the dataset summarises a single player performance on a given target-map. Every map played is expected to produce ten player-map rows, though some missing metadata and anomalous group sizes were found and are reported in Section 4.1. Each row also contains map-stat identifiers giving match and map context. The variable `match_id` is extracted from HLTV's `mapstatsid` and identifies an individual map-stat page, not the best-of-three series it may come from. Table 3.1 summarises the main groups.

Table 3.1: Main raw data groups collected from public HLTV player-map pages.

Group	Examples
Player and map-stat identifiers	Player name, map-stat identifiers, source URL, event and date
Team and opponent context	Player team, opponent team, result and series type where available
Map context	Map name, team rounds, opponent rounds, half scores, starting side and overtime flag
Player statistics	Kills, deaths, assists, flash assists, headshot kills, ADR, rating, opening kills, opening deaths, multi-kills, traded deaths, and clutch count fields

The added depth given by map-level collection over series-level collection was because player and team performance can vary by map. Counter-Strike series use a pick-ban process to decide which maps the teams play on. Petri et al. (2021) showed that the map selection decision process can be modelled and can affect the estimated win probability. This supports the decision to treat target-map identity and map-specific history as potentially useful information. The availability feature tiers are able to examine whether the knowledge of the target-map improves this project's individual-performance prediction.

The raw player statistics are outcome variables for the map that has just been played. They are useful for constructing the target and for building historical features, but they are not valid input features for the same row. For example, the current map's rating is used to define whether the row is a good match, but it is not supplied to the classifier as an input.

3.3 Data Cleaning and Target Construction

Before feature engineering and model training, the raw HLTV dataset had to be cleaned and turned into the final dataset. The focus here is to ensure that the data is clean and has minimal risk of target and temporal leakage.

Numeric and categorical values were standardised by being converted into consistent formats. Rows missing the fields `player`, `date`, or `rating` were all removed as those fields were required for ordering and target construction. Remaining missing numeric values were handled by median imputation inside the preprocessing pipeline, whilst missing categorical values, such as `starting_side`, had their missing values assigned "Unknown". Any duplicate player-map rows were identified through searching for identical `player`, `match_id`, and `map` identifiers and then removed. Rows were then sorted by `player`, `date`, and deterministic tie-break fields. This is all done within the feature pipeline.

Missing `map`, `series_type`, and `match_id` values and rows grouped by map-stat identifiers were counted to compare with the expected count of ten per map. This was not part of

`validate_feature_dataset` but described the generated dataset. The data-quality counts are reported in Section 4.1.

The target was defined at a player-map level, not a player-match level. Series would differ between best of one matches and best of three matches, so instead the data was looked at on a per-map basis. The main classification target can be defined as:

$$\text{good_match} = \begin{cases} 1, & \text{if player rating} > 1.0, \\ 0, & \text{otherwise} \end{cases} \quad (3.1)$$

Each row is assigned a binary label of either `good_match` or not based entirely on their rating, the label is not a perfectly accurate descriptor of good play. A captain/IGL (In-Game Leader) can provide impact that is not seen in statistics, through superior strategising and mid-round calling. The rating threshold could be changed, and this would produce different class distribution and model behaviours.

Regression was done as a secondary continuous-rating task despite the original aim being that of binary classification with the prediction being simply good or not good. The continuous rating value was directly predicted and kept as `target_rating` to test whether using a binary target obscured a stronger relationship. This is not the primary research question.

3.4 Feature Engineering

The raw post-map statistics could not be used directly as inputs for the same row because they describe post-map outcomes and would cause data leakage. Feature engineering converted the post-map data from past player-map records into usable features describing form, workload, map experience, opponent context, and team context.

Controls were added to reduce temporal leakage. The rows were sorted chronologically using deterministic tie-break fields before the history-based features were calculated. Rolling and expanding calculations used `shift(1)` to exclude current-row outcomes. Current-row outcomes were all included in the list of excluded values that were to not be included in the modelling export. Historical team-map aggregates were constructed in a separate chronological table, shifted, before being merged back into the dataset. Whilst these controls reduce leakage risk, they do not prove the models are entirely leakage free.

Features were organised into three feature-availability tiers. The HLTV post-match statistics pages are retrospective and provide both map and starting-side information, but a live version of this project would require a separate pre-match source. Map identity and starting side were therefore kept in separate tiers because their availability differs with different tournament and event procedures. Table 3.2 summarises the assumptions made by the three feature-availability tiers.

Player-form features existed as a combination of short-term and longer-term historical features, such as `avg_rating_prev_5` which described recent form, `career_avg_rating_prev` which represented broader player history, and `trend_rating` which compared a change in recent performance. Player workload was represented by features such as `days_since_last_map` and `maps_last_14_days`. The code also flagged back-to-back maps or player breaks longer than two weeks.

Player team and opponent context was constructed by looking at earlier team-map aggregates. Examples of these include `team_rating_prev_5`, `opp_team_rating_prev_5`, and their difference, `team_rating_gap_prev_5`. Player familiarity on specific maps was represented in features such as `avg_rating_prev_5_on_map` and `maps_played_on_map_prev`.

Table 3.2: Feature availability tiers and the assumptions made of available information.

Tier	Assumption and examples
Core pre-map	Player history, recent load, recent form, opponent and team history and contexts. Numeric examples: <code>avg_rating_prev_5</code> , <code>ewm_rating</code> , <code>matches_played_prev</code> , <code>days_since_last_map</code> , <code>team_rating_prev_5</code> and <code>opp_team_rating_prev_5</code> . Categorical examples: <code>player</code> , <code>player_team</code> , <code>opponent</code> , <code>series_type</code>
Map-known	Features that require knowledge of the current map. Examples: <code>avg_rating_prev_3_on_map</code> , <code>avg_rating_prev_5_on_map</code> , <code>maps_played_on_map_prev</code> , <code>map_winrate_prev</code> , <code>map_adr_prev_5</code> , <code>map_kdr_prev_5</code> , <code>map_opening_duel_ratio_prev_5</code> . The map itself is also added as a categorical feature.
Starting-side-known	Features that require knowledge of the starting side. An example is <code>avg_rating_prev_5_on_side</code> . <code>starting_side</code> is also added as a categorical feature.

Team, opponent, and map histories could all be limited, and sparse histories can lead to unstable averages. A `prior_weight` of 5 was implemented to shrink selected estimates towards broader player history. This reduces the influence of limited histories but does not solve the underlying issue. Overtime features were also included to describe the previous performance on maps that did or did not face overtime, they are listed in Appendix A.

Current-map player rating, kills, deaths, and other post-map outcomes were explicitly never used as model inputs. They were only used to construct the targets or shifted histories. Appendix A contains a list of all the final input groups.

3.5 Modelling Design

The modelling task was to help reach the aim of the project, to answer the question of if predicting whether a player will have a good match on a target map is possible. To achieve this, the main task is binary classification of `good_match`. The classification output was the probability for the positive class. For comparison, a secondary regression task existed that predicted `target_rating`, this was not the main project objective. The reason behind the secondary regression task was to test if the binary target lost too much information from the continuous rating. Table 3.3 summarises the candidate set used during validation.

All stochastic candidates used seed 42 and were intentionally only lightly tuned. The soft vote is the unweighted mean of the validation-selected Random Forest and Extra Trees probabilities.

The feature availability tiers, mentioned in Section 3.4, were used here in the modelling. By using these availability tiers, `core_pre_map`, `map_known`, and `starting_side_known`, the project was able to compare the results from scenarios with more or less conservative context. The core tier used historical features available before the target map without knowledge of target-map identity or starting side. The `map_known` tier assumes the target map is known at prediction time. The same goes for the `starting_side_known` tier, it assumes the starting side is known at prediction time. These tiers are explicitly defined in the modelling code.

Table 3.3: Model families and configurations evaluated during validation.

Model family	Evaluated configuration
Logistic regression	One-hot categorical features; Median imputation and standardisation; <code>max_iter=1500</code> ; <code>random_state=42</code>
Random Forest classifiers	150 trees; leaf sizes 20, 50 and 100; <code>max_features="sqrt"</code>
Extra Trees classifiers	150 trees; leaf sizes 20 and 50; numeric and selected target-encoded variants
Gradient Boosting classifiers	120 estimators; learning rate 0.05; depths 2 and 3; target-encoded starting-side-known features
Regression	Ridge with <code>alpha = 1.0</code> ; Random Forest/Extra Trees with 120 trees and leaves 20/50; Gradient Boosting with 120 estimators, learning rate 0.05, and depths 2/3

The simplest classification baseline used was a dummy majority classifier that only predicts the most frequent class. This showed the baseline level of performance achievable without learning meaningful patterns. Logistic regression provided a simple linear classification baseline which gives a simpler comparison against tree models. For the secondary regression experiment, a dummy mean regressor predicted the mean rating and Ridge regression was used to provide a regularised linear regression baseline for predicting continuous rating.

As player performance could possibly be affected by non-linear interactions between features, tests were done using Tree ensemble models. An example would be a player's recent form mattering differently when facing opponents of differing strengths or with differing map familiarity. For classification, Random Forests (Breiman, 2001), Extra Trees (Geurts et al., 2006) and Gradient Boosting (Friedman, 2001) classifiers were tested, specifically the Gradient Boosting classifiers in target encoded classification experiments. Additionally, regression versions of Random Forest, Extra Trees, and Gradient Boosting were tested. These were not heavily optimised, as only a small set of candidate configurations were tested. The Random Forest and Extra trees candidates used different minimum leaf sizes, and the Gradient Boosting candidates used shallow tree depths.

The linear models used both numeric and categorical features. The numeric features were median imputed and scaled whereas the categorical features were imputed with "Unknown" and one-hot encoded. Tree models that used `feature_mode="numeric"` used engineered numeric features with median imputation. This was done to avoid creating a high dimensional one-hot representation for player, team, and opponent names.

Selected experiments used smoothed target encoding for categorical columns such as `player`, `player_team`, `opponent`, `event`, `series_type`, `map`, and `starting_side`. During the model selection process, category statistics were fitted on the training data and applied to the validation rows. The encoder would be refitted on the combined training and validation periods after candidate selection and applied to the test period. Test targets were never used to construct encodings. If a category was unseen then it would be filled with the corresponding training global mean, and a smoothing value of 50 shrank rare category estimates towards that mean.

The training implementation was not out-of-fold. The same training observations contributed to the training category means that were used to encode itself. This did not expose validation or test targets but it could make training representations look more optimistic than they really are. Cross-fitted or forward-chain target encoding would be a stricter future implementation (Micci-Barreca, 2001; Pargent et al., 2022).

A simple soft voting ensemble was tested for classification. This averaged the predicted probabilities from the strongest candidates of Random Forest and Extra Trees. The reason this was included was to test for any complementary signals captured by two different tree ensemble methods. This was simple by design and did not train a new model.

3.6 Train, Validation and Test Split

The experiments used a chronological train/validation/test split, apart from the all-player baseline script which used a chronological train/test split only. The split was based on unique timestamps so that rows that shared a timestamp would remain on the same side of a boundary. Only the one chronological division was used. The later chronological test period began on 7 August 2024 and contained 14,940 rows. It had been previously examined in the baseline experiment. The extended candidate models and the classification threshold were both selected using validation results. For the extended experiments, the earlier data was split into 37,169 training rows and 10,901 validation rows with validation beginning on 26 February 2023. The results may depend on the particular period chosen. The code uses `TEST_SIZE = 0.2` and `VALIDATION_SIZE = 0.2` to choose the cut-off dates for the splits, the 20% splits were based on unique timestamps rather than 20% of the rows. In a live setting, the model would be trained on all available past maps and then used to predict future maps that haven't happened yet. Rolling or expanding temporal validation may have provided a stronger estimate across multiple eras ([Cerqueira et al., 2020](#)).

Due to the aim of this project being to predict future player-map performance, a chronological split was naturally chosen. Choosing a random split would lead to older and newer rows being mixed in the train and test splits which could make the evaluation less realistic as player, team, and opponent form changes over time and so does the game itself. Updates to the game over time can bring balance changes that affect the game in small or large ways. As this is the case, chronological evaluation is better aligned with this project's research question.

The splits were not player isolated, so the same player could appear in training, validation, and test splits if they were active during those periods. This is not an issue as the whole aim of the project is to use a player's historical performance to predict their future performance. Going with an unseen player split would instead answer the question of whether the model can generalise to predict performance of a player with no history. The design is intended to look at the history of the player, team, and opponents to predict the player's future.

3.7 Evaluation Metrics

The extended classification experiments were primarily compared using validation ROC-AUC, which was chosen due to the classifier outputting probabilities and not just class labels. ROC-AUC is able to measure how high positive examples are ranked above negative examples. Another benefit of ROC-AUC was that it is able to be used without a chosen threshold as it is threshold independent. This was a benefit as it can help to detect predictive signal within the engineered features. Sorting classification results by ROC-AUC will show the top validation classifiers and allow candidates to be selected for the final test evaluation.

Additional metrics were also reported as ROC-AUC is not always easy to interpret. ROC-AUC assess the model's ranking ability and does not directly assess probability calibration. AUC of 0.630 does not mean the model is 63% accurate. The additional metrics included accuracy, balanced accuracy, recall, precision, F1, and confusion matrix counts. No confidence interval was calculated. These are all easier to interpret as they each describe actual predicted

labels. These threshold-dependant metrics and confusion counts are suitable for describing operational class-label behaviour. The code reports `tn`, `fp`, `fn`, and `tp` for classification models. The default threshold was set to 0.5 for extended experiments, with additional thresholds being selected based on validation data for F1 and balanced accuracy. The threshold search was between 0.30 and 0.70. Threshold-dependent metrics are needed to describe behaviour at the selected operating threshold.

The regression models predicted continuous `target_rating` and were evaluated with MAE, RMSE, and R^2 . MAE describes the average absolute error in rating units. RMSE describes rating error too but more heavily penalises larger errors. R^2 compares the model predictions against a mean rating baseline. The R^2 being low means the model explains a small amount of variance, whilst a negative R^2 would mean it actively performs worse than the baseline of predicting the mean rating. RMSE was used to rank validation regressors by the extended experiment script to assist in selection of regressors for final testing.

Because one map typically has ten individual player rows, observations within a map are correlated. Future uncertainty analysis should group the observations by map-stat identifier to take advantage of this.

3.8 Validation Checks

Validation checks were implemented into the pipeline to reduce any chance of leakage or mistakes in the feature engineering process. The checks were made after construction by having the feature script call `validate_feature_dataset` before writing the processed features. A summary of the implemented checks can be found in Table 3.4.

Table 3.4: Validation checks used during feature dataset construction.

Check	Purpose
Expected features check	Checked for the presence of all expected feature columns
Target rule check	Checked that <code>good_match</code> followed the target rule set by Eq. 3.1.
Regression target check	Checked that <code>target_rating</code> matches the original rating value.
Player first row check	Checked that each player's first row has <code>matches_played_prev</code> equal to zero.
Shifted history check	Checked that selected historical features were missing from first player rows.
Leakage column check	Checked that current-map team aggregates such as <code>team_rating</code> and <code>team_adr</code> are not present in the modelling export.
Rolling average check	Checked accuracy by independently recalculating a sample <code>avg_rating_prev_5</code> value from the previous five ratings.
Team history check	Checked accuracy by independently recalculating <code>team_rating_prev_5</code> and compared it with the engineered value.

These checks are there to catch any obvious mistakes in the parts of the pipeline most sensitive to leakage, they do not fully ensure that the model is useful nor do they replace

evaluation. The checks alone cannot prove there is no leakage within the models. The statistical performance itself is evaluated within Chapter 4. Validation checks help give the model credibility, as any leakage could artificially inflate the model's performance (Kapoor and Narayanan, 2023).

3.9 Implementation and Reproducibility

The project was originally entirely within notebooks, but eventually moved on to being a small Python project. Reusable code was located within `src/cs2_predictive_analysis`. Within `features.py` were the target constants such as `good_match` and `target_rating`. It also contained the feature group definitions, the feature engineering functions, and the validation checks. Structuring the project this way helped to avoid duplicate instances of logic across notebooks and also made the pipeline easier to run.

The project used scripts to regenerate the main processed artefacts. Using these scripts would be the main reproducibility route as they regenerate data and tables. Feature construction used pandas and NumPy. The model preprocessing, estimators, and evaluation metrics all used scikit-learn (Pedregosa et al., 2011). Table 3.5 summarises the inputs, outputs, and checks.

Table 3.5: Project components and scripts.

Component/Script	Input	Output	Purpose and validation
<code>features.py</code>	Raw player-map records	Engineered feature dataframe	Constructs targets and shifted histories. Checks for required fields and historical ordering.
<code>build_features.py</code>	<code>maps_from_cache.csv</code>	Processed feature CSVs	Handles generation. Calls validation checks before writing.
<code>train_all_players_model.py</code>	Model-feature CSV	Baseline-results CSV	Runs the dummy, logistic, and Random Forest baselines using the chronological split.
<code>experiment_models.py</code>	Model-feature CSV	Classification, regression, and selected-model results CSVs	Performs validation-based candidate and threshold selection and the final test evaluation.
Notebooks	Processed data and saved results	Exploratory and reporting outputs	Records of earlier exploration.

The notebooks were the original pipeline and were mainly used for initial exploration and reporting. The feature engineering notebook allowed for a walkthrough of the process. There was also the baseline modelling notebook that was used for early model development and results. The all player modelling notebook was created when the process switched from training on a specific player to training on all players, it summarised the larger modelling results for reporting. After moving to scripts, the notebooks still remained useful as they communicated the process effectively.

Some limitations come from the fact exact reproduction relies on HLTV's public web database. HLTV's structure and database can change or become unavailable over time. Exact reproduction also requires the raw HLTV cache.

3.10 Social, Legal and Ethical Considerations

The project used statistics and information from public professional Counter-Strike matches. The project collected public professional-match statistics and player aliases. No contact details, private communications, surveys, or experiments with participants, and no biometric, private, or other special-category data was collected. The dataset was built around HLTV's public match records, rather than direct subject research. The risk to identifiable players was substantially reduced by not relying on any private or non-public data. Risk still remains due to data relating to identifiable professional players being processed and so results must be carefully reported as they are not evaluations of a player's overall worth.

The main legal issue comes from the responsibility of using public data. The source of the data, HLTV, should be clearly acknowledged and it should not be implied that the project contains endorsement from HLTV, nor should it be presented as an official HLTV product. The data scraping was handled with care, caching was used as to reduce load by avoiding repeatedly requesting the same pages and the downloader had built-in delays between requests to further reduce load. The caching is also useful for reproducibility due to the nature of public data and its ability to change or become unavailable.

The social issues within this project come from the data being used to predict player performance, which can affect how people judge or view individual players. The public statistics themselves don't accurately reflect the impact support roles and in-game leaders have as they do not fully reflect the impact that teamplay, communication, utility usage, and team strategy have on a match's outcome. As is the case, a player given a lower predicted probability is not necessarily a bad player. Therefore the model should be treated as merely an experiment and not an accurate reflection of a player's quality. Due to the model predicting if a player will have a `good_match` or not, this project holds risk of gambling related interpretations. Because of this, the report avoids claims of guaranteed prediction and report represents the results as modest and uncertain. No betting odds or betting recommendations were produced. It also avoids predicting the outcome of a match directly, by predicting the performance of an individual player and not the team.

Table 3.6: Social, legal and ethical risks, implemented mitigations and residual limitations.

Risk	Possible impact	Mitigation and residual limitation
Predicting identifiable-player outcomes	Unfair judgement or damage to reputation	Use cautious language and ensure transparency of limitations when presenting results. There is valuable missing context
Scraping load on data source	Disruptions to source service	Built-in delays, page caching, and exponential back-off retry. No endorsement is claimed
Temporal and data leakage	Inflated performance results	Shifted histories, chronological splits, explicit disclosure.
Gambling and betting misuse	Model outputs used for betting	No odds or betting recommendations. Individual predictions can still be repurposed.
Source schema drift	Failure to parse future pages	Cached pages are retained. Live collection still requires unchanged public structure.

The project work was entirely desk based, none of the experiments involved were physical and there was no participant interaction. Health and safety risks were limited as only computer desk risks such as prolonged screen time and poor ergonomics were present. The information security risk was also low as the project did not contain any sensitive data such as credentials, private communications, or other private data. The data was stored as locally cached web pages and processed CSV files.

3.11 Chapter Summary

This chapter explained the project as an applied machine learning pipeline built on public HLTV player-map data. Historical features were engineered to reduce leakage by using shifted calculations. The modelling experiments used chronological splitting so more recent maps could be used as future data for evaluation. Validation checks took place to check for mistakes and leakage within the features.

Chapter 4

Results

4.1 Dataset Summary

The processed modelling dataset had a total of 63,010 player-map rows combined from data from 1,795 professional players ranging from the 14th of January 2016 to the 1st of March 2026. Overall, the positive class rate for `good_match` was 52.8% meaning the class distribution was close to balanced. The final chronological test period data ranged from the 7th of August 2024 to the 1st of March 2026 and contained 14,940 rows. Table 4.1 summarises the dataset.

Table 4.1: Processed dataset summary.

Name	Value
Player-map rows	63,010
Unique Professionals	1,795
Earliest date	14/01/2016
Most recent date	01/03/2026
Positive class rate	52.8%
Test period date	07/08/2024
Test period rows	14,940

10,190 rows (16.17%) of the 63,010 observation rows were without parsed map and series metadata. Out of the 14,940 test-period observation rows, 620 rows (4.15%) lacked map metadata. These missing values were dealt with by the preprocessing pipeline through the categorical preprocessing pipeline's unknown-value handling. There were also ten map-stat identifiers with unexpected group sizes of eleven player rows rather than the expected ten, and ten rows completely lacked a map-stat identifier. This weakened the interpretation of the `map_known` tier.

The modelling export excluded columns that contained current map outcomes from the input features. Excluded columns such as, `rating`, `adr`, `kills`, `deaths`, `kast`, or any values influenced from the result were only used to construct historical shifted features and to define targets. `target_rating` was only kept as the regression outcome.

The processed dataset contains data from both the CS:GO and CS2 eras of Counter-Strike.

4.2 All-Player Baseline Models

This section reports on the first all-player baseline results. Originally the project was modelling based on a single player history, the newer method uses the full dataset. The baseline models were evaluated using the final chronological test set, using train/test and not train/validation/test. The three model types used were dummy majority classifier, logistic regression, and a numeric-only Random Forest. The purpose of these models was to see if historical features were able to perform better than a no-skill baseline. Table 4.2 shows the main baseline train/test results.

Table 4.2: Baseline all-player classification results on the chronological test set.

Model	Acc.	Bal. acc.	Precision	F1	ROC-AUC
Dummy majority	0.529	0.500	0.529	0.692	0.500
Logistic regression, starting-side-known	0.561	0.559	0.584	0.585	0.590
Random Forest numeric, map-known	0.593	0.590	0.609	0.627	0.626
Random Forest numeric, starting-side-known	0.593	0.589	0.607	0.630	0.626

The dummy majority classifier predicts the most frequent class, in this case the positive class is slightly more common and so `good_match` is predicted for every row. This explains how F1 is high for the dummy majority despite it not having any useful predictive ability. The balanced accuracy and ROC-AUC are both 0.500, which shows that it does not in fact separate good performances from non-good performances. This is why the project took multiple metrics into account, as on its own, F1 is misleading for this baseline.

Logistic regression is an improvement over the dummy baseline, its ROC-AUC is 0.590 in the starting-side-known result which suggests the features contain some predictive signal, however limited. The numeric-only Random Forest baselines outperform logistic regression, its map-known and starting-side-known Random Forest results both rounded to 0.626 ROC-AUC which is consistent with, but does not prove, the presence of non-linear relationships within the historical features that were not captured by the linear baseline. Although there is an improvement, it is not drastic.

The accuracy only changes moderately between models, partly due to the classes being close to balanced and the task being noisy. ROC-AUC gives a better view of the model's ability to rank good performances above non-good performances, and balanced accuracy helps describe the model's ability to perform on both classes rather than just favouring the majority. The baseline results are able to justify the extended experiments in Section 4.3.

4.3 Extended Classification Experiments

The extended classification experiments introduced a new validation process to the train/test split where models were first evaluated using their validation ROC-AUC for candidate selection. After selection, the models were further evaluated using the same chronological test set. The chronological test set had been previously examined during baseline development. The best validation configuration chosen was a soft voting ensemble that averaged probabilities from the best Random Forest and Extra Trees candidates using their validation ROC-AUC. The selected ensemble achieved validation ROC-AUC 0.651 and test ROC-AUC 0.630, showing that the validation performance was partly carried over to the test set. There was an overall

gain over the baseline Random Forest, but it was small. Table 4.3 shows the classification results.

Table 4.3: Selected extended classification experiment results with comparisons.

Model	Thresh.	Val. AUC	Test AUC	Test bal. acc.	Test F1
Dummy majority	0.50	0.500	0.500	0.500	0.692
Random Forest numeric, map-known, leaf 50	0.50	0.648	0.627	0.592	0.631
Random Forest numeric, starting-side-known, leaf 20	0.50	0.648	0.626	0.589	0.630
Soft-vote ensemble, F1 threshold	0.38	0.651	0.630	0.526	0.692
Selected soft-vote ensemble	0.50	0.651	0.630	0.592	0.636
Selected soft-vote ensemble, balanced-accuracy threshold	0.52	0.651	0.630	0.596	0.611

With a threshold of 0.50, over all 14,940 test rows, the output had 5,258 true positives and 3,653 true negatives, but also 3,386 false positives and 2,643 false negatives. The confusion matrix is shown in Table 4.4. This shows that the model correctly identifies many of the good maps, but also makes many false negative and positive mistakes.

Table 4.4: Selected soft-vote classifier confusion matrix at threshold 0.50.

	Predicted not good	Predicted good
Actual not good	3,653	3,386
Actual good	2,643	5,258

The threshold selection changed predicted labels and affected balanced accuracy, precision, recall, F1, and confusion-matrix counts. It did not affect ROC–AUC as that is calculated from probability ranking and not a fixed threshold. Lowering the threshold would cause more positive case predictions which can increase recall and F1 but can also reduce balance between the classes. The balanced accuracy threshold aimed to create a more even compromise between the positive and negative classes.

The strongest classification result was a test ROC–AUC of 0.630 which is a minor improvement over the Random Forest baseline’s result of 0.627. The extended workflow showed that ROC–AUC was above the 0.5 baseline on the chronological evaluation period, but the signal is very limited.

4.4 Regression Results

The regression experiment was a secondary experiment and did not focus on the main research question. It predicted a continuous `target_rating` instead of the binary `good_match`. The purpose of the regression experiment was to test whether predicting the original rating would reveal a stronger signal than the binary classification task had. The regressors were selected using validation RMSE, the results can be seen in Table 4.5. The regressor with the best validation RMSE was a Random Forest regressor with numeric feature mode, the starting-side-known feature set, and a minimum leaf size of 50. On the chronological test set, the

selected Random Forest achieved test MAE 0.272, RMSE 0.351, and R^2 0.083. Some other candidates had a fractionally higher test R^2 but were not selected using test performance. Most models' test results were approximately $R^2 = 0.08$, so the difference was almost negligible.

Table 4.5: Selected rating regression results on the chronological test set.

Model	Validation RMSE	Test MAE	Test RMSE	Test R^2
Random Forest regressor, starting-side-known, leaf 50	0.329	0.272	0.351	0.083
Random Forest regressor, starting-side-known, leaf 20	0.329	0.272	0.351	0.084
Random Forest regressor, map-known, leaf 50	0.329	0.272	0.351	0.084
Random Forest regressor, map-known, leaf 20	0.329	0.273	0.351	0.082
Random Forest regressor, core pre-map, leaf 50	0.330	0.272	0.351	0.083

The top validation regressors all had very close results and were all Random Forest numeric models. Most of the models rounded to similar scores of MAE $\approx$ 0.272, RMSE 0.351, and R^2 0.08. The fact they all had such similar results provides little evidence of a material improvement from the map-known and starting-side-known feature tiers.

MAE 0.272 shows the average absolute rating error was around 0.27 rating points, RMSE 0.351 shows that larger errors still existed, and R^2 0.08 means the model explained a small share of the variation in player-map rating. The R^2 being positive shows that the model was better than predicting the mean rating. Most of the variation remained unexplained though which further supports the notion that signal does exist but it is limited. This is not a massive surprise, as a player's rating on a single map can be affected by many things that are not yet captured within public statistics.

4.5 Summary of Findings

The results are used cautiously to answer the research question here. Historical HLTV player-map statistics were able to predict `good_match` with limited ranking performance above the baseline, but the gap between the model and the baseline is not spectacular. This shows the extended validation/ensemble methodology helped, although it was not a great improvement. Because the results were only limited, the classifier should be interpreted as a weak ranking model rather than a predictor with guaranteed outcomes.

Regression was experimented with, but the signal found was not much stronger. The validation-selected regressor achieved test MAE 0.272, RMSE 0.351, and R^2 0.083. The R^2 being so low showed that most of the variation in map rating was unexplained.

The conclusion is that the public HLTV historical statistics provide limited evidence of ranking signal when evaluated against the selected test period. The signal found is not enough on its own for strong individual player-map rating predictions.

Chapter 5

Discussion and Analysis

5.1 Interpretation of the Predictive Signal

The strongest classification result came from the selected soft-vote ensemble and was a test ROC–AUC of 0.630. This is higher than the dummy baseline’s ROC–AUC 0.500 which provides limited evidence on the selected test period that the model is not equivalent to guessing randomly and that the historical features do include some predictive signal. This suggests the ranking performance outperforms the baseline on the selected test period.

Whilst a showing the existence of predictive signal, ROC–AUC 0.630 is far from strong and many positive and negative cases overlap. This can be seen in the results confusion matrix, Table 4.4, where there were many false positives and false negatives. As the model is not a strong predictor, it is useful for weak ranking and should not be considered close to confident.

Since the research question asked whether public historical statistics contain predictive value, the results have provided a positive but limited answer. The results show that there is measurable predictive value within the historical data. The modest results are still worthwhile as the evaluation reduces the risk of leakage and there are clearly understood limitations. Had there been a model containing leakage with much stronger results, it would not be valuable or meaningful.

The public HLTV statistics are not enough on their own for reliable case-level predictions of upcoming player-map performance. Many aspects influence a player’s upcoming performance, such as roles, tactics, mid-map events, roster changes, psychological tilt, preparation, and meta changes, most of which not in HLTV’s database. Some of what affects a player’s performance is measured within HLTV’s statistics, but too much of what affects a player’s performance is not currently captured. The result is modest ranking separation.

5.2 Why Prediction is Difficult

The outcome of an individual player’s performance on a Counter-Strike map is difficult to predict as there are many interacting factors in play. Historical features such as rating, ADR, KDR, and similar variables describe the outcome of previous maps but don’t explain how those variables came to be. A strong rating in a player’s recent game could be the result of improved individual form, weaker opponents, taking on a favourable role, strong team and utility support, comfort on a map, and many other short term variations (Xenopoulos et al., 2020). The public HLTV data is unable to completely separate these influences and is thus unable to fully explain performance.

A major variable missing from the model’s features are the player’s roles within a team.

Each role within a team comes with different expectations and duties. Star riflers, AWPers, entry fraggers, anchors, supports, and in-game leaders are all important to a team's success, but come with different statistical expectations. Whereas star riflers and AWPers often have impact that is more visible in rating based statistics, roles such as the IGL or support have their impact captured less within the same statistics. Meaningful actions taken by these players such as utility usage, trading, calling, anchoring, or setting up the star players, are not reflected in the statistics and this weakens the model (Xenopoulos and Silva, 2022). As the model does not have reliable role labels, the model must infer a player's role indirectly from their history, team context, and statistics.

Counter-Strike is a team game and therefore a player's individual performance can be massively influenced by the team's situation round to round (Zhang et al., 2022). The player-map data does not contain much of the game's mid-map contexts. Economy, role, tactics, roster state, and preparation all affect individual player performance and are not included within the dataset. A player can have unusually poor performance due to being put in difficult situations by their team such as having unfavourable fights without support or having their team die in untradeable positions. All these unrecorded factors mean the model only sees the outcome of historical games and has to infer the reason behind these outcomes rather than see them.

The dataset also contains data from both Counter-Strike: Global Offensive and Counter-Strike 2. Many features and balances have been both introduced and taken out of the game in its longstanding history. There have been many changes within the period the dataset covers, changes to the map pool were made yearly, weapon balance changes were made, economy changes were made. Most notable is the switch from CS:GO to CS2 and a new engine that called for a "remaster" of old maps. Some maps merely had improved textures and some maps had entire reworks. With each of these changes, no matter how small, the meta is forced to adapt. While the changes to the meta may seem miniscule update to update, jumping from the oldest entry in the dataset to the most recent would see a drastic change to the meta. Because of this, patterns learned on old data might not transfer to newer matches (Chitayat et al., 2023). The chronological test split helps as the final evaluation is on the most recent period. This does not fix the issue entirely though and more explicit update-aware or era-aware features would be required to model this better.

The model includes some opponent and map history but is not aware of any preparation either team made. The model misses out on information such as the veto order, the reasoning behind a map pick, how much practice a team has had outside of officials, any anti-stratting, recent roster changes, if there are stand-ins present, if a team's coach is not present, if a player has recently changed roles, or any new playbook changes. These factors being unrecorded likely contributes to the prediction errors and the low regression R^2 .

5.3 Classification and Regression Compared

Both the classification and regression experiments were able to support the same conclusion. The classification of `good_match` performed only modestly above the baseline with the strongest result reaching test ROC-AUC 0.630. Rating regression predicted continuous `target_rating`, the validation-selected regressor achieved $R^2 = 0.083$, suggesting that approximately 8.3% of the variation in player-map rating was explained relative to the test-set mean baseline. The vast majority of variation was left unexplained. The conclusion reached by experimenting with regression was that binary classification was not holding the project back from a much stronger continuous relationship.

The binary classification target, `good_match`, remained useful due to its simplicity and

interpretability. The threshold rating > 1.0 directly matched the project's main question of whether a player is likely to have a good upcoming map. For this reason, the classification output is much easier to explain than a continuous rating prediction. A probability of `good_match` is also easier to interpret as probabilistic ranking signal rather than a precise rating forecast. Therefore, classification was more appropriate for the project's research question.

There were still limitations of the threshold as the binary target does lose information. A rating of 1.01 and 1.40 are both treated as positive, and 0.99 is treated as negative even though it is very close to 1.01. The rating threshold is applied uniformly across roles and players and a good map for a player that averages 0.50 rating might still be under the threshold. Future work could look for improvement by including ordinal targets such as poor/average/good/excellent, role specific thresholds, a mixture of regression and classification, or uncertainty bands around the threshold.

5.4 Was the Dataset Large Enough?

By row count, the dataset was not small. It contained a total of 63,010 player-map observations. This provided enough observations to fit the evaluated all-player tabular models. The original single-player approach had a much more limited dataset as individual players have far fewer map records than the full dataset. The all-player approach improved the methodology as it was able to combine all individual training examples into one large dataset and allowed the models to learn general patterns that exist across players whilst also lowering the dependence the models had on a single player's history.

The number of rows in a dataset is not the only metric of it being sufficient. Each row was still a public summary of a single player-map performance and missed out on important context. The context missing from the rows included player roles, economy states, tactics, roster changes, preparation, practice, and the current meta. Some players, especially players newer to the professional scene, had sparse histories and some players also moved from roster to roster frequently. Features such as opponent and team strength would also continuously change over time due to constant changes to team situations. These issues all combined to reduce the effectiveness of the predictive information available within the features.

The conclusion is that the dataset was sufficient for an exploratory machine learning study to fit and evaluate the selected models and come to the conclusion that some predictive signal does exist within the dataset. However, despite the size of the dataset, the information contained within was not rich enough to produce a strong predictive performance. Whilst adding more rows may increase stability, adding richer features could give a larger improvement to the models.

5.5 Model Behaviour and Feature Design

The tree-based models were shown to outperform the logistic regression baseline. The Random Forest baseline improved ROC-AUC to around 0.627 could be interpreted as some of the signal potentially being non-linear and based on interactions from a combination of features. For example, a player's recent form matters differently depending on the strength of their opponent or a player's current workload might matter differently when they are unfamiliar on the map they are going to be playing on. Random Forests and Extra Trees are suited to tabular data as they are able to split on combinations of features without the need for a manually specified equation.

The gap between Random Forest and ensemble was small as the selected soft-vote ensemble reached a test ROC–AUC around 0.630 and the strongest Random Forest was around 0.627. This was an improvement. The gap suggests that adding model complexity only helped slightly meaning that the model choice alone may not overcome the limitations, but the public features that were currently available to the project may have been insufficient. Continuing to improve to more complex methods could possibly increase the results further, but might not be enough on its own as the observed small gain suggests no large jump could come from tweaking the model choice. This further supports the idea that richer data is what is required for further improvements.

The selected ensemble contained both a numeric Random Forest and a target-encoded Extra Trees model. Whilst being part of the selected configuration, the independent contribution of target encoding is unable to be quantified. The target-encoding design took steps to avoid leakage. The lack of out-of-fold training encodings is also a methodological limitation (Pargent et al., 2022).

The design choice behind having feature tiers was to separate the assumptions about the knowledge available at prediction time. `core_pre_map` was the safest tier and used information that can reliably be known further before the target map. `map_known` and `starting_side_known` both assumed the availability of knowledge that is typically available very close to the time of the target map, but still before the map. Separating these tiers allowed the project to check how the extra context affects or improves the results. The experiments showed the differences in feature tiers was small, suggesting that the additional context available to `map_known` and `starting_side_known` did not cause a large jump in model performance.

The model is best understood when treated as one analytical signal. It could be useful to analysts for helping flag players or maps for a closer review but should not replace domain judgement. Analysts using this as a tool would still be required to use their own judgement and take into account the context these models do not capture. The model's value to analysts exists in highlighting weak statistical tendencies but is not suitable to be considered a stand-alone decision maker for analysts.

5.6 Limitations and Threats to Validity

This section explains the main threats to the project's validity. Limitations within the project affect how strongly the limited predictive signal shown in the results is able to be interpreted. Limitations do not just exist as performance issues for the models but can also affect what the target means, how reliable the input data is, and whether the results are able to be applied outside of the project setting. The main issues facing this project are summarised in Table 5.1.

The conclusions made using the `map_known` tier are weakened by a number of rows not containing a known map due to having missing map metadata. Unknown-category handling allowed the rows with missing map data to be processed and remain in the dataset despite having no parsed map context. Several anomalous maps had records for eleven players instead of the expected ten. These issues add data-source uncertainty. They do not invalidate the map-tier interpretation, but they do weaken it.

Internal validity concerns whether the observed results reflect the intended modelling setup or if they were caused by leakage, data, or implementation mistakes (Kapoor and Narayanan, 2023). The project strengthened its internal validity with many leakage controls throughout. Historical outcome-derived features were shifted to avoid any target map leakage, any statistics related to current map outcomes were strictly excluded from inputs, the dataset was split chronologically, target encoding used training data only, and the feature pipeline contained

Table 5.1: Main limitations and validity threats.

Limitation	Effect on interpretation
Only public data	Important mid-map, role, and tactical data is not available on HLTV, limiting predictive performance.
Rating target	Rating is a metric constructed by HLTV that is used in post-map summaries and may not capture every valuable contribution a player can make. The evolution of rating over time means it may not represent the same construct across eras.
Shared threshold for all roles	good_match may be harder for some roles to achieve than others, such as support or IGL roles.
Long date span	The dataset ranges over a significant period of time. Both CS:GO and CS2 are included in this range. Patterns learned on older data may not carry over to more recent CS2 performance prediction.
Prior test-period examination	The baseline stage examined this period before extended model evaluation happened. The final test results should therefore be treated as indicative evidence instead of a fully independent estimate.
Limited histories	Some players or teams may have sparse historical records, leading to noisy estimates.
Public source dependent	The pipeline depends on public HLTV pages which may change or become unavailable over time.
Missing or anomalous parsed metadata	Some rows had missing map-stat context. Some rows with map-stat information had more than the expected ten players. These rows weakened the map-tier interpretability.
Only one temporal split	Performance could possibly depend on the chosen era. Stability was not measured on other chronological periods.
Target encoding not out-of-fold	Training rows were used in their own category encodings, potentially inflating the training results.
No grouped confidence intervals	Each map typically contained ten player-map observations so row-level uncertainty would be too narrow. Statistical uncertainty around AUC was not estimated at the map level.

validation checks as part of its process. Although these intentional design choices serve to make the results more credible by significantly reducing the risk of leakage inflating the results, they do not prove the model is perfect or entirely leakage free.

External validity is limited. Since the model was trained on public HLTV data from professional matches, the model should not automatically be expected to carry over to other forms of Counter-Strike such as matchmaking/Premier, FACEIT, ESEA, lower tier scenes, or even other similar esports like Valorant. Professional Counter-Strike matches have much more advanced preparation and are increasingly using dedicated analysts to perform structured game reviews. This added support surrounding the professional players highlights the substantial differences between professional Counter-Strike and conventional matchmaking, FACEIT, and even low-tier esports. This limits the model’s transferability when being used outside of a professional environment. The analysts could likely also have access to private data relating to tactics, roles, preparation, and player condition that is not included on HLTV (Xenopoulos et al., 2022).

Another key issue is construct validity. The research question is about player performance and the target is defined as $\text{rating} > 1.0$. Rating was chosen as it is easily interpretable. Rating is not a perfect proxy though and underrepresents many aspects of team play and support roles. This can be seen as the model not predicting whether a player is going to perform

well in a broad sense but instead predicting whether the player is going to exceed the rating threshold defined as `good_match`. Another issue with rating is that the formula behind rating has changed over the course of this dataset's covered time period. The formula was updating to "Rating 2.0" in 2017, "Rating 2.1" in 2024, and "Rating 3.0" in 2025 (Milovanovic, 2017; Richards, 2024, 2025). Whilst the threshold `rating > 1.0` is consistent, what it represents will not be precisely the same across all eras.

Data-source validity is another issue to consider. The source of this project's data, HLTV, is public. It may change structure or become unavailable at any point meaning live data may clash with currently cached data's format causing parsing errors or live data may become unobtainable. Caching all scraped data was important for reproducibility as all the data used to train the models has been preserved.

5.7 Ethical Interpretation

This project makes predictions about identifiable professional players. Even though the inputs come from a public source, any predictions made should be carefully and honestly described as being uncertain and limited. The model should not be used to label players as either good or bad and only predicts whether a future player-map rating may exceed the rating threshold used for `good_match`. The threshold being shared across all player roles may systematically disadvantage the predictions given to players in support or IGL roles. Many aspects of a player's value are not captured by the inputs or threshold, and players should not be reduced to the predicted probability of a single rating statistic. Data minimisation and purpose limitation remain relevant even for public-source data (Information Commissioner's Office, n.d.a,n).

The model also makes frequent errors, as seen in Table 4.4, and so should be presented as being uncertain. False positives and negatives could both be used to unfairly affect perceptions of a player if taken at face value. A low predicted probability should not be treated as proof that a player will perform poorly in the target map, and a high predicted probability should not be treated as a guarantee of upcoming good map performance. The output should be interpreted as weak statistical tendencies. The output should not be used as a standalone decision maker for scouting, roster changes, or player valuation without human review accompanied with richer context.

Esports performance prediction often contains connections to betting or gambling. This project used no betting odds and was intentionally designed to not be used for betting recommendations. Predicting individual player performance was a clear design choice over predicting the winner of a target map as it less directly aligned with betting contexts, although there still remains a risk of player performance predictions being misused in that context. The modest classification ROC-AUC and low regression R^2 should further discourage misuse of this project within gambling. Further steps to distance this project from gambling are taken such as presenting this project as an exploratory analysis tool rather than a standalone decision making system. Throughout the project emphasis is put on the project's uncertainty, missing features, limited predictive strength, academic purpose, and lack of any guarantee in the predicted outcome. The outcome should be presented with uncertainty and the intended academic purpose.

5.8 Overall Answer to the Research Question

The final answer to the research question is that the experiments provided limited evidence of ranking performance above the baseline on the chronological split. The project showed

that historical public HLTV statistics showed measurable predictive value. The project did not provide accurate individual forecasting or causal relationships. The final result of ROC–AUC 0.630 is a measurable improvement over the dummy majority, but is still modest in absolute terms as the model still contained many false positives and false negatives. The credibility of the results can be reinforced by the chronological evaluation design which aimed to reduce the risk of the models being able to look ahead.

Regression was experimented with and the results supported the same conclusion. The validation-selected regressor result had $R^2 = 0.083$, with all leading candidates achieving close to $R^2 = 0.08$. This meant there was a slight predictive signal in the available features but that the majority of the rating variance was left unexplained.

The project does not show that player-map performances can be accurately forecast from public statistics alone. Many unrecorded aspects are missing from the dataset that limit the available signal.

Chapter 6

Conclusions and Future Work

6.1 Conclusion

This project's research question asked whether a professional Counter-Strike player's historical match statistics can be used to predict their upcoming map performance. The experiments provided a qualified positive answer, the models achieved a modest result above the baseline when predicting future professional player-map performance using historical data. There is a measurable, but weak, signal. It is not nearly strong enough for confident predictions. The specific aim of the project was to be able to predict whether a player's rating in an upcoming target map would exceed 1.0.

The project implemented and evaluated its technical aim by building a machine learning pipeline that focused on reproducibility and reduction of leakage risk. The pipeline featured the scraping, parsing, and cleaning of data before it was used for machine learning. The pipeline collected public professional map data by scraping the HLTV database and caching all results locally. It would then parse and clean the data collected. The target would then be constructed and the historical features would be engineered. Once complete, the models would be trained and evaluated. To aid in reducing the pipeline's leakage risk, any target-map outcome statistics were excluded from inputs and used only to define targets and create shifted historical features for later rows. The pipeline used a chronological split so the model would train on historical data and then be tested on the latest data. The target encoding was fitted on the training and validation set in the extended experiments, and never on the test set. Validation checks were built into the pipeline's feature generation to ensure no obvious mistakes slipped through before processed features were written.

The project started with a single-player modelling approach. The project moved to an all-player modelling approach to overcome some of the limitations that came from the single-player approach. By training the models on every player in the dataset, instead of just one individual, it allowed for fuller usage of the dataset. The resulting model would likely be more general and less dependent on a single player's history. The strongest result achieved within this project came from classification, specifically the selected soft-vote ensemble. It achieved a test ROC-AUC approximately 0.630, clearly above the dummy baseline and also clearly above logistic regression. The ensemble, however, was only slightly above the strongest Random Forest baseline. Regression was used in experiments to see if it was also difficult to predict exact rating. The regression experiments showed that it is also difficult to predict as the best score achieved was a very low R^2 around 0.08.

The historical statistics are able to capture the player's recent form, their long term history, their experience on certain maps, and is also partially able to capture their workload, some team context, and some opponent context. Some of what is not covered is player roles and

additional team contexts such as strategy, economy management, preparation, roster changes, or player condition. These missing variables have likely contributed to the weak signal.

Despite the weak signal, the project has still produced a useful empirical result. It features a structured and re-runnable pipeline from retained inputs designed to reduce leakage that is able to illustrate a realistic estimate of what public statistics can achieve when used to estimate a player's future map performance. A model with leakage or a model trained on a non-chronological split could possibly result in better metrics than this project's models, but their results would be untrustworthy.

This project showed that the public historical HLTV statistics produced above-baseline performance in this experiment, even if it was weak. For this project to achieve a stronger signal, collecting richer contextual data would be a worthwhile direction to explore.

6.2 Future Work

The most important change that can be made for future work would be the collection of richer data. Future work could look at parsing professional demos into round-level data ([Xenopoulos and Silva, 2022](#)). The round-level data could be used to make features that reflect the state of the economy, force/eco/full-buy statistics, clutch attempts, retake situations, player positioning, trade opportunities and success rate, and the location of kills and deaths. These features would aid in better describing the context behind a player's performance, and would likely contain more useful signal than just map-level statistics do. It would also benefit the model to include the roles a player takes on for each individual map. Future work could also give a fairer prediction to support and IGL players by adding thresholds specific to each role or even by separating models by role. These changes could also contribute to better construct validity for future work.

Future work could also expand upon the data collected for team contexts. Potentially useful features could include a team's global ranking, the strength of their recent opponents, how stable their roster is, how long their current team have been together, whether there are stand-ins or whether the coach is absent ([Zhang et al., 2022](#)). Data regarding the map veto process could also be useful as it can describe whether a map was forced upon a team, if they picked it, if they typically ban it, and how that compares with their opponent's map picks ([Petri et al., 2021](#)). Further features could include the stage of the tournament they are in, whether the event is online or LAN, how far the players have had to travel, and how drastic of a time zone change there is, where the data is reliably available.

The modelling stage could also be expanded upon but it should not be the main focus in future work. Calibration analysis could be useful as it can show whether the predicted probabilities matched observed frequencies. It could, for example, look at when the model predicts 60% probability and show if around 60% of those maps become `good_match`. Future work could use rolling or repeated temporal evaluation in the evaluation stage to test performance across multiple chronological splits ([Cerqueira et al., 2020](#)). Model interpretation methods could be added. Methods such as permutation importance, partial dependence, SHAP values, and feature ablation could all help identify the feature groups that contribute the most signal.

Future work could also develop this project into an analysis dashboard ([Xenopoulos et al., 2022](#)). It would not suddenly be able to give guaranteed predictions or be able to replace analyst judgement. Having outputs such as the predicted `good_match` probability, recent trends, team contexts, explanations of important features, and warnings about missing data and sparse histories would allow the dashboard to find the most use as a tool used to help analysts ask better questions.

Chapter 7

Reflection

7.1 Project Development

The project went through significant changes during its development. The original modelling approach focused on individual player histories as a means of predicting that same individual's future map performance. This was a naive approach as it seemed easier to visualise, seemed more focused, and matched the research idea of predicting an upcoming performance. It quickly became apparent that this approach would be too narrow for the purposes of this research project.

The single-player approach had problems. By looking at only the data of one individual, only a small part of the dataset was being utilised. Some players, especially newer ones, had too few recorded maps played to support stable individual model training. The single-player approach's results were highly dependent on the player that was chosen. These factors made it difficult to generalise and meant the model was less able to learn patterns with broader relationships.

Moving to all-player modelling was one of the most important decisions in this project's development. Doing this allowed for the project to use the full dataset and gave the models far more player examples to learn patterns from. The methodological improvement was not enough for the model to produce a strong predictive performance. This showed that the single-player approach had not been the only limitation of the project and that even with data from all players being taken into account, the public HLTV statistics did not produce a strong enough signal.

7.2 Learning About Data Leakage

Throughout the project's development, the importance of avoiding leakage was taken into account for many of the major design choices. Any feature that is calculated using data available only after the target map has been played must not be used within the prediction for the target map itself. The data can be used when predicting later maps, but if it were used to predict the target map then it could severely skew the results to look more positive than they are. It is better to have a lower scoring prediction that can be validated, than a more positive prediction that cannot be validated.

Rolling and expanding features might look overly useful if calculated incorrectly. If they contain data that can only be known after the map has been played, then they skew the results. To avoid this, the engineered historical calculations were shifted and only used data from previous rows. Target map outcome statistics were not entirely useless outside of being

used for later rows' historical features though as they were used to define `good_match` and to keep `target_rating` for regression. When fitting target encoding, steps were also taken to reduce leakage. During model selection, target encoding was fitted on training data and applied to validation data. For final testing, they were refitted on combined training-plus-validation data and applied to the test data. Thanks to the chronological split, the test data was the most recent data, and the training data was only in the past relative to the test data. Test targets were never used. If the project had used random splitting then the rows inside the training and test splits would have been mixed with both older and newer rows, making prediction less realistic.

Leakage prevention measures were less exciting to implement as training new models was, but it was one of the most important aspects of this project. These measures directly contribute to the credibility of this project's results. This project's design has reflected the notion that methodology is more important than simply chasing a higher score. In future ML projects I would take more steps to define what is known at prediction time and ensure validation measures are implemented from the start.

7.3 Reproducibility and Scope Control

Another valuable lesson that this project taught me was how useful it is to move reusable code out of notebooks. Notebooks were useful early on in the project's life and were especially useful for exploring different methodologies, as well as for breaking the process into steps, allowing for an easier explanation of the process. The workflow quickly outgrew the notebooks, and they actively became a source of friction when rerunning the project consistently. To get around this hurdle, reusable logic was moved into Python scripts and package files. This improved reproducibility and the feature engineering and modelling scripts made it clearer where modelling results came from and made the final report easier to explain.

The final structure had scripts become the main way to reproduce the project. The scripts regenerated the processed feature files, the modelling datasets, as well as the results CSVs.

Scope control became important as there were many possible experiments that could have been done. It was tempting to continuously add different experiments, for example testing variants of models, features, thresholds, or target encoding. Some experimentation was required, and much of the experimentation was useful. The experimentation that was done showed that tree-based models outperformed simple logistic regression and that target-encoded candidates appeared among the stronger validation results in some experiments. Experimentation also led the project to its strongest result, produced by the soft-vote ensemble. Further experimentation started to give diminishing returns. If there were improvements, they were minor. There was a risk of taking too much time away from important steps in the project such as validation, explanation, interpretation, and the writing of the report. Once the project had a clear and reproducible set of results, the choice had to be made to stop further experimentation. This was the better choice in terms of project management.

7.4 Personal Evaluation

The most satisfying part of this project came from being able to turn a broad and personal interest in Counter-Strike into a focused research project. Being able to apply my learned computer science knowledge to a field I have great interest in has been very insightful and enjoyable. My interest in this project came from an interest in Counter-Strike statistics. The research question was a specific and testable question, whether historical player-map

data could predict future player-map performance. The final result was not dramatic as no high-confidence predictions were produced, but the project did show that there was some predictive value in historical data. The answer given was valuable and supported by careful feature engineering and leakage-aware choices.

If I were to start this project over, I would define what is and what is not available at prediction time as one of the first steps in the process. I would also define the project's leakage rules and establish validation checks from the start. I would also plan the report structure much earlier and ensure that all scripts and results were created with the final report in mind. This would have significantly reduced the amount of reworking required in the later stages of the project.

This project improved through repeated iteration. The final version of the project was clearer about its limitations, more reproducible, and had a more defensible methodology. The project taught me that good ML work is not just achieved by selecting algorithms. The lessons learned from my time creating this project apply beyond this dataset.

References

- Breiman, L. (2001), 'Random forests', *Machine Learning* **45**(1), 5–32.
- Bunker, R. P. and Thabtah, F. (2019), 'A machine learning framework for sport result prediction', *Applied Computing and Informatics* **15**(1), 27–33.
- Cerqueira, V., Torgo, L. and Mozetic, I. (2020), 'Evaluating time series forecasting models: An empirical study on performance estimation methods', *Machine Learning* **109**(11), 1997–2028.
URL: <https://link.springer.com/article/10.1007/s10994-020-05910-7>
- Chitayat, A. P., Block, F., Walker, J. and Drachen, A. (2023), 'Beyond the meta: Leveraging game design parameters for patch-agnostic esports analytics', *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* **19**(1), 116–125.
URL: <https://ojs.aaai.org/index.php/AIIDE/article/view/27507>
- De Bois, M., Parmentier, F., Puget, R., Tanti, M. and Peltier, J. (2025), 'PandaSkill – player performance and skill rating in esports: Application to league of legends'.
URL: <https://arxiv.org/abs/2501.10049>
- Friedman, J. H. (2001), 'Greedy function approximation: A gradient boosting machine', *The Annals of Statistics* **29**(5), 1189–1232.
- Geurts, P., Ernst, D. and Wehenkel, L. (2006), 'Extremely randomized trees', *Machine Learning* **63**(1), 3–42.
- Hamari, J. and Sjöblom, M. (2017), 'What is esports and why do people watch it?', *Internet Research* **27**(2), 211–232.
- HLTV.org (2026), 'Counter-strike statistics database'. (Accessed: 3 May 2026).
URL: <https://www.hltv.org/stats>
- Hodge, V. J., Devlin, S. M., Sephton, N. J., Block, F. O., Cowling, P. I. and Drachen, A. (2021), 'Win prediction in multi-player esports: Live professional match prediction', *IEEE Transactions on Games* **13**(4), 368–379.
URL: <https://pure.york.ac.uk/portal/en/publications/win-prediction-in-multi-player-esports-live-professional-match-pr/>
- Information Commissioner's Office (n.d.a), 'Principle (c): Data minimisation'. (Accessed: 12 May 2026).
URL: <https://web.archive.org/web/20260429161753/https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-protection-principles/a-guide-to-the-data-protection-principles/data-minimisation/>

- Information Commissioner's Office (n.d.b), 'What is personal data?'. (Accessed: 12 May 2026).
URL: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/personal-information-what-is-it/what-is-personal-data/what-is-personal-data/>
- Kapoor, S. and Narayanan, A. (2023), 'Leakage and the reproducibility crisis in machine-learning-based science', *Patterns* **4**(9), 100804.
URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10499856/>
- Micci-Barreca, D. (2001), A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems, in 'ACM SIGKDD Explorations Newsletter', Vol. 3, pp. 27–32.
URL: <https://dl.acm.org/doi/10.1145/507533.507538>
- Milovanovic, P. T. (2017), 'Introducing rating 2.0'. (Accessed: 7 May 2026).
URL: <https://www.hltv.org/news/20695/introducing-rating-20>
- Pargent, F., Pfisterer, F., Thomas, J. and Bischl, B. (2022), 'Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features', *Computational Statistics* **37**(5), 2671–2692.
URL: <https://link.springer.com/article/10.1007/s00180-022-01207-6>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011), 'Scikit-learn: Machine learning in python', *Journal of Machine Learning Research* **12**, 2825–2830.
- Petri, G., Stanley, M. H., Hon, A. B., Dong, A., Xenopoulos, P. and Silva, C. (2021), 'Bandit modeling of map selection in counter-strike: Global offensive'.
URL: <https://arxiv.org/abs/2106.08888>
- Richards, H. N. (2024), 'Introducing rating 2.1'. (Accessed: 7 May 2026).
URL: <https://www.hltv.org/news/40051/introducing-rating-21>
- Richards, H. N. (2025), 'Introducing rating 3.0'. (Accessed: 7 May 2026).
URL: <https://www.hltv.org/news/42485/introducing-rating-30>
- Smerdov, A., Burnaev, E., Somov, A. and Stepanov, A. (2020), 'Ai-enabled prediction of esports player performance using the data from heterogeneous sensors'.
URL: <https://arxiv.org/abs/2012.03491>
- Valve (2026), 'Counter-strike 2'. (Accessed: 1 May 2026).
URL: <https://www.counter-strike.net/cs2>
- Xenopoulos, P., Doraiswamy, H. and Silva, C. (2020), Valuing player actions in counter-strike: Global offensive, in '2020 IEEE International Conference on Big Data', pp. 1283–1292.
URL: <https://arxiv.org/abs/2011.01324>
- Xenopoulos, P., Rulff, J. and Cláudio, S. (2022), 'ggviz: Accelerating large-scale esports game analysis', *Proceedings of the ACM on Human-Computer Interaction* **6**(CHI PLAY), 1–22.
URL: <https://arxiv.org/abs/2107.06495>
- Xenopoulos, P. and Silva, C. (2022), 'Esta: An esports trajectory and action dataset'.
URL: <https://arxiv.org/abs/2209.09861>

- Zhang, W., Muric, G. and Ferrara, E. (2022), 'Individual and collective performance deteriorate in a new team: A case study of cs:go tournaments'.
URL: <https://arxiv.org/abs/2205.09693>

Appendix A

Feature Groups

This appendix lists the model inputs produced by `scripts/build_features.py`.

A.1 Core Pre-Map Features

The core numeric inputs were

- **History and workload:** `matches_played_prev`, `days_since_last_map`, `maps_last_7_days`, `maps_last_14_days`, `avg_days_between_maps_prev_10`, `back_to_back_flag`, `long_break_flag`.
- **"Good match" history:** `pct_good_matches_prev_5`, `pct_good_matches_prev_10`, `career_good_rate_prev`, `streak_good_matches`, `streak_bad_matches`, `maps_since_last_good_map`.
- **Overtime history:** `ot_rate_prev_10`, `avg_rating_prev_ot`, `avg_rating_prev_non_ot`.
- **Rating form and variation:** `avg_rating_prev_3`, `avg_rating_prev_5`, `avg_rating_prev_10`, `avg_rating_prev_20`, `career_avg_rating_prev`, `ewm_rating`, `rolling_std_rating_prev_5`, `trend_rating`, `delta_rating_last_vs_prev_5`, `max_rating_prev_5`, `min_rating_prev_5`, `best_minus_avg_rating_prev_10`.
- **ADR and KDR history:** `avg_adr_prev_5`, `avg_adr_prev_20`, `trend_adr`, `delta_adr_last_vs_prev_5`, `rolling_std_adr_prev_5`, `avg_kdr_prev_5`, `avg_kdr_prev_20`, `trend_kdr`, `delta_kdr_last_vs_prev_5`, `rolling_std_kdr_prev_5`.
- **Opening-duel history:** `avg_opening_duel_ratio_prev_5`, `trend_opening_duel_ratio`.
- **Player against opponent:**
- **Opponent allowance:**
- **Team and opponent context:** `team_rating_prev_5`, `team_adr_prev_5`, `team_rating_gap_prev_5`, `team_adr_gap_prev_5`, `team_map_rating_prev_5`, `team_map_rating_gap_prev_5`, `opp_team_rating_prev_5`, `opp_team_adr_prev_5`, `player_vs_opp_team_gap_prev_5`, `opp_team_map_rating_prev_5`, `player_adr_vs_opp_team_gap_prev_5`, `player_minus_team_rating_prev_5`, `avg_rating_vs_opp_team`, `shrunk_avg_rating_vs_opp_team`, `maps_played_vs_opp_team`, `opp_allowed_rating`.

```
prev_10, opp_allowed_good_rate_prev_10, opp_allowed_rating_career_prev,  
opp_allowed_good_rate_career_prev,
```

The core categorical inputs were `player`, `player_team`, `opponent`, and `series_type`. Selected target-encoding experiments used `event`.

A.2 Map-Known Features

The map-known tier added the target-map category `map` as well as:

- `avg_rating_prev_3_on_map`, `avg_rating_prev_5_on_map`;
- `maps_played_on_map_prev`, `map_winrate_prev`;
- `map_rating_gap`, `map_adr_prev_5`, `map_kdr_prev_5`;
- `map_opening_duel_ratio_prev_5`, `shrunk_avg_rating_prev_5_on_map`.

A.3 Starting-Side-Known Features

The starting-side-known tier added the categorical `starting_side` and numeric `avg_rating_prev_5_on_side`.